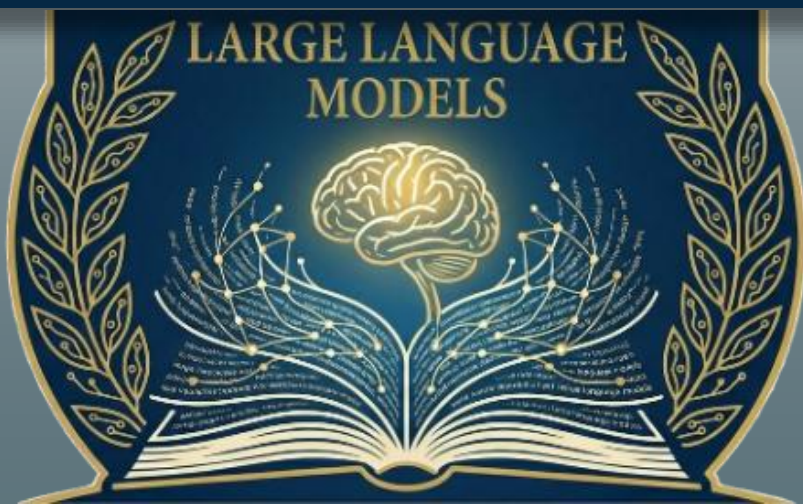


Integrating Knowledge & Retrieval Augmented Generation (RAG)



Mohammad Javad Dousti

Sources:

- Mohit Iyyer's Advanced NLP's slides from UMass.
- Ken Shi's Natural Language Computing slides from U of Toronto.
- Y. Gao et al. "Retrieval-augmented generation for large language models: A survey." *arXiv preprint arXiv:2312.10997* (2023).

Integrating Knowledge

Overview

- ❑ Recap: language models (LMs)
- ❑ What does a LM know?
- ❑ Techniques to add knowledge to LMs
 - Use an external memory

Recap: LMs

- **Standard language models** predict the **next word** in a sequence of text and can compute the probability of a sequence

The students opened their _____ **books** _____.

- **Masked language models** (e.g., BERT) instead predict a **masked token** in a sequence of text using bidirectional context

went **store**
I [MASK] to the [MASK].

- Both types of language models can be trained over large amounts of unlabeled text!

Recap: LMs

- ❑ Traditionally, LMs are used for many tasks involving **generating** or **evaluating the probability** of text:
 - Summarization
 - Dialogue
 - Autocompletion
 - Machine translation
 - Fluency evaluation
 - ...
- ❑ Can a language model be used as a **knowledge base**?

What does a language model know? (1)

- ❑ iPod Touch is produced by_____.
- ❑ London Jazz Festival is located in_____.
- ❑ Dani Alves plays with_____.
- ❑ Carl III used to communicate in_____.
- ❑ Ravens can_____.

Examples taken from [Petroni et al., EMNLP 2019](#) to test BERT-Large.

What does a language model know? (2)

- ❑ iPod Touch is produced by___Apple_____.
- ❑ London Jazz Festival is located in__London_____.
- ❑ Dani Alves plays with___Santos_____.
- ❑ Carl III used to communicate in___German_____.
- ❑ Ravens can__fly_____.

Examples taken from [Petroni et al., EMNLP 2019](#) to test BERT-Large.

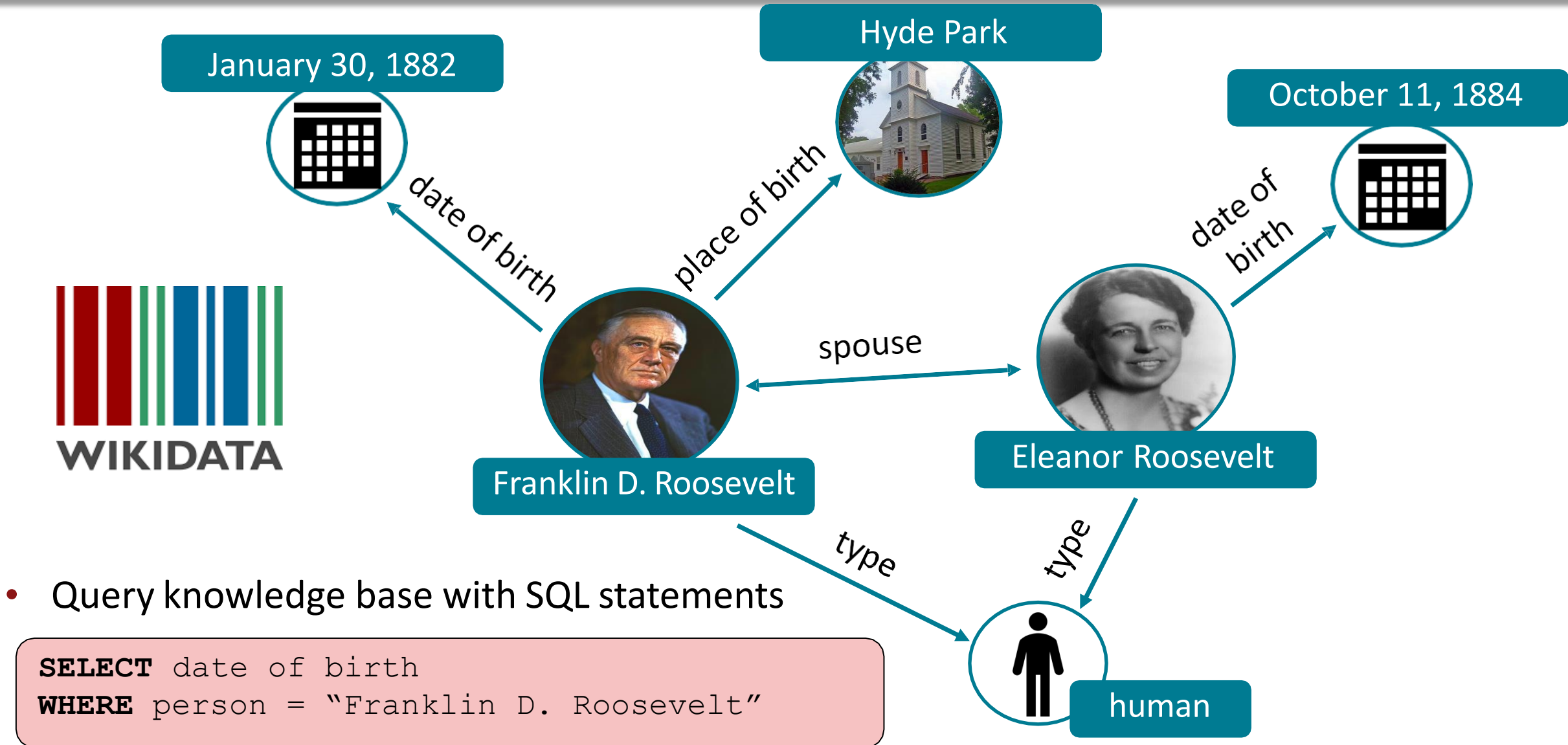
What does a language model know? (3)

- ❑ Takeaway: Predictions generally make sense (e.g. the correct types), but **are not all factually correct**.
- ❑ Why might this happen?
 - **Unseen facts**: some facts may not have occurred in the training corpora at all
 - **Rare facts**: LM hasn't seen enough examples during training to memorize the fact
 - **Model sensitivity**: LM may have seen the fact during training, but is sensitive to the phrasing of the prompt
 - Correctly answers “*x was made in y*” templates but not “*x was created in y*”
- ❑ The **inability to reliably recall knowledge** is a key challenge facing LMs today!
 - Recent works have found LMs can recover some knowledge, but have a way to go.

The importance of knowledge-aware language models

- ❑ Stretch goal: Can LMs ultimately **replace traditional knowledge bases**?
 - Instead of querying a knowledge base for a fact (e.g. with SQL), query the LM with a natural language prompt!
 - Of course, this requires LM to have high quality on recalling facts

Querying traditional knowledge bases



ChatGPT 3.5 ▾



You

When was Franklin D. Roosevelt born?



ChatGPT

Franklin D. Roosevelt was born on January 30, 1882.



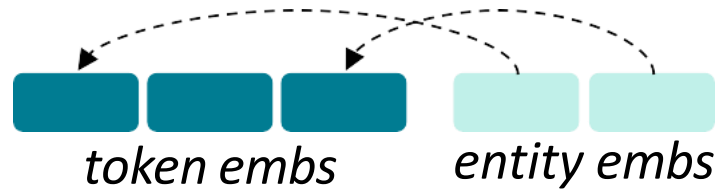
Is this conversation helpful so far?



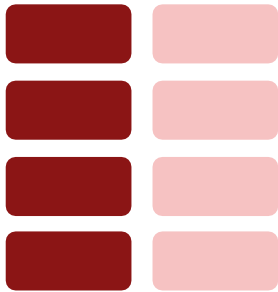
Advantages of language models over traditional KBs

- ❑ LMs are pretrained over large amounts of **unstructured and unlabeled text**
 - KBs require manual annotation or complex NLP pipelines to populate
- ❑ LMs support more **flexible natural language queries**
 - Example: *What does the final F in the song U.F.O.F. stand for?*
 - Traditional KB wouldn't have a field for "final F"; LM may learn this
- ❑ However, there are also many open challenges to using LMs as KBs:
 - **Hard to interpret** (i.e., why does the LM produce an answer)
 - **Hard to trust** (i.e., the LM may produce a realistic, incorrect answer)
 - **Hard to modify** (i.e., not easy to remove or update knowledge in the LM)

Techniques to add knowledge to LMs



keys values



Add pretrained entity embeddings

- ERNIE (Enhanced Language Representation with Informative Entities - by Baidu)
- KnowBERT (Knowledge Enhanced BERT - by AllenAI)

Use an external memory

- KGLM (Knowledge Graph Language Model)
- kNN-LM (k-Nearest Neighbors Language Model)

Modify the training data

- WKLM (Weakly Supervised Knowledge Pretrained Language Model)
- ERNIE (another!), SSM (Salient Span Masking)

Method 2: Use an external memory

- ❑ Give the model access to an external memory (a key-value store with access to KG triples or context information)
- ❑ **Advantages**
 - Can better support injecting and updating factual knowledge
 - Often without more pretraining!
 - More interpretable

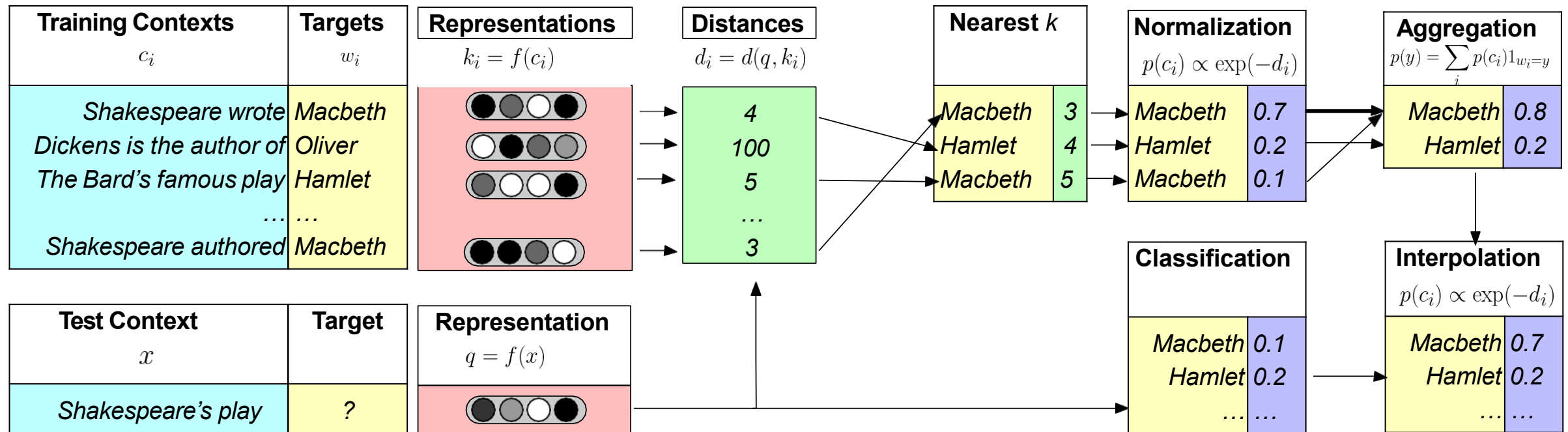
Nearest Neighbor Language Models (kNN-LM)

- Key idea: **learning similarities between text sequences** is easier than predicting the next word
 - Example: “Dickens is the author of _____” $\approx$ “Dickens wrote _____”
 - Qualitatively, researchers find this is especially true for “long-tail patterns”, such as rare facts
- So, store all representations of text sequences in a nearest neighbor datastore!
- At inference:
 - Find the k most similar sequences of text in the datastore
 - Retrieve the corresponding values (i.e. the next word) for the k sequences
 - Combine the **kNN probabilities** and **LM probabilities** for the final prediction
$$P(y|x) = \lambda P_{kNN}(y|x) + (1 - \lambda) P_{LM}(y|x)$$

λ is a tuned hyperparameter

[Khandelwal et al., ICLR 2020]

More recent takes: Nearest Neighbor Language Models (kNN-LM)



Retrieval Augmented Generation (RAG)

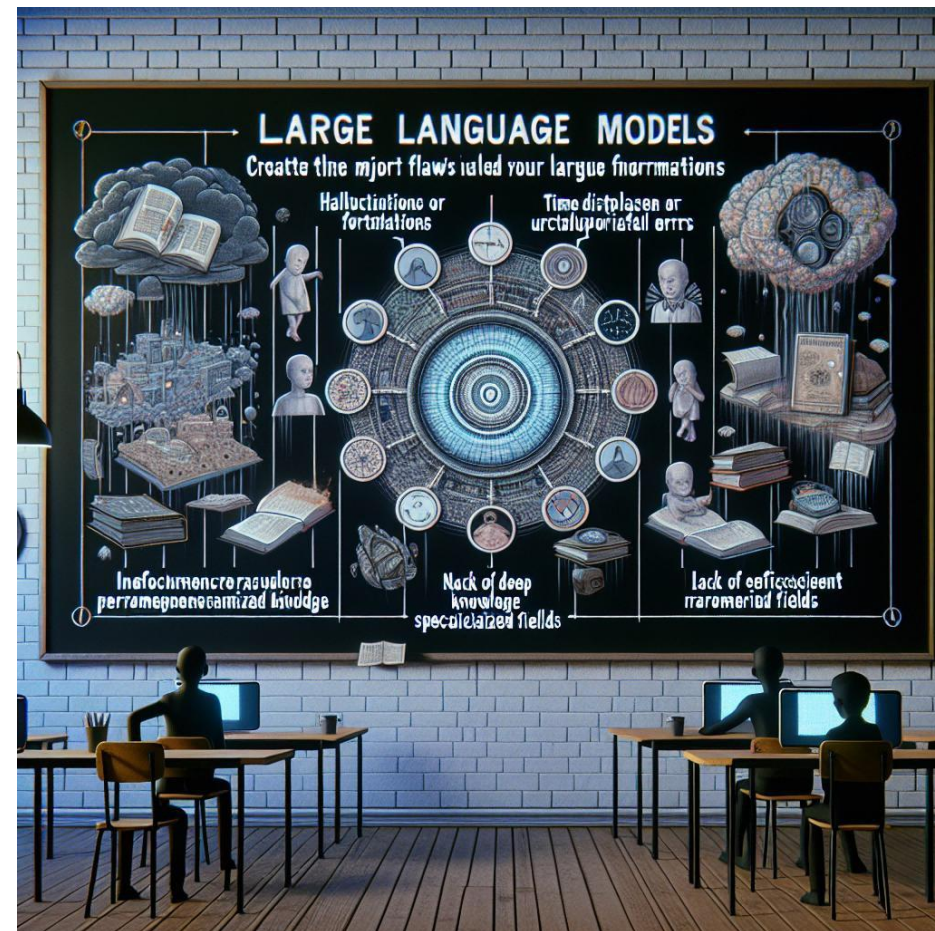
Background

❑ Drawbacks of LLMs

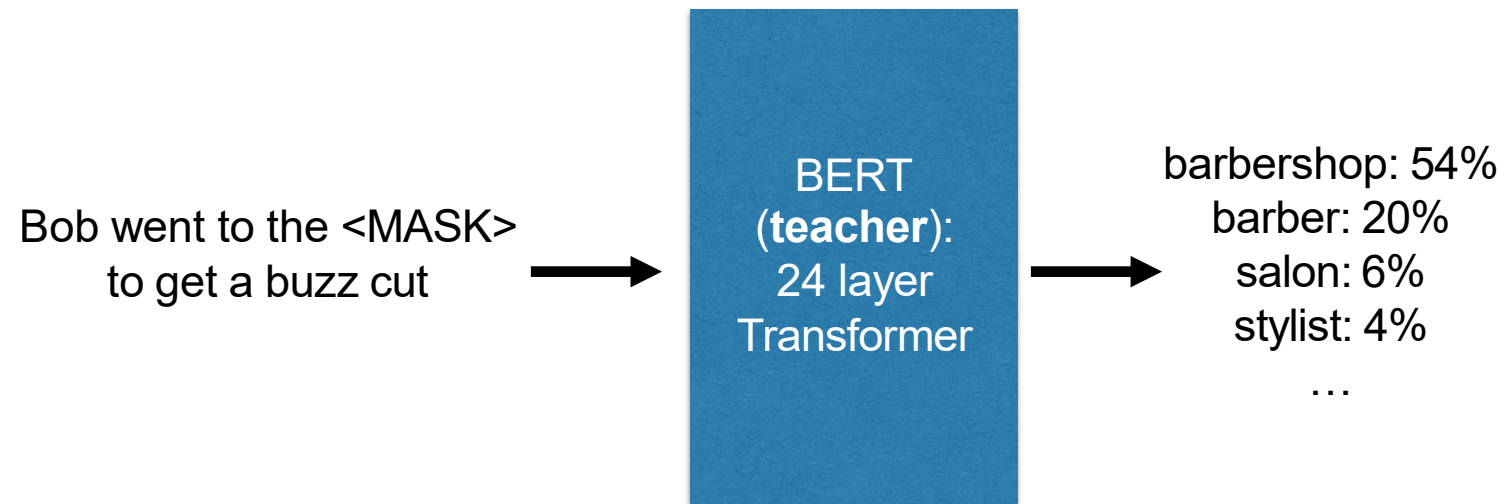
- Hallucination
- Outdated information
- Low efficiency in parameterizing knowledge
- Lack of in-depth knowledge in specialized domains
- Weak inferential capabilities

❑ Practical Requirements of Application

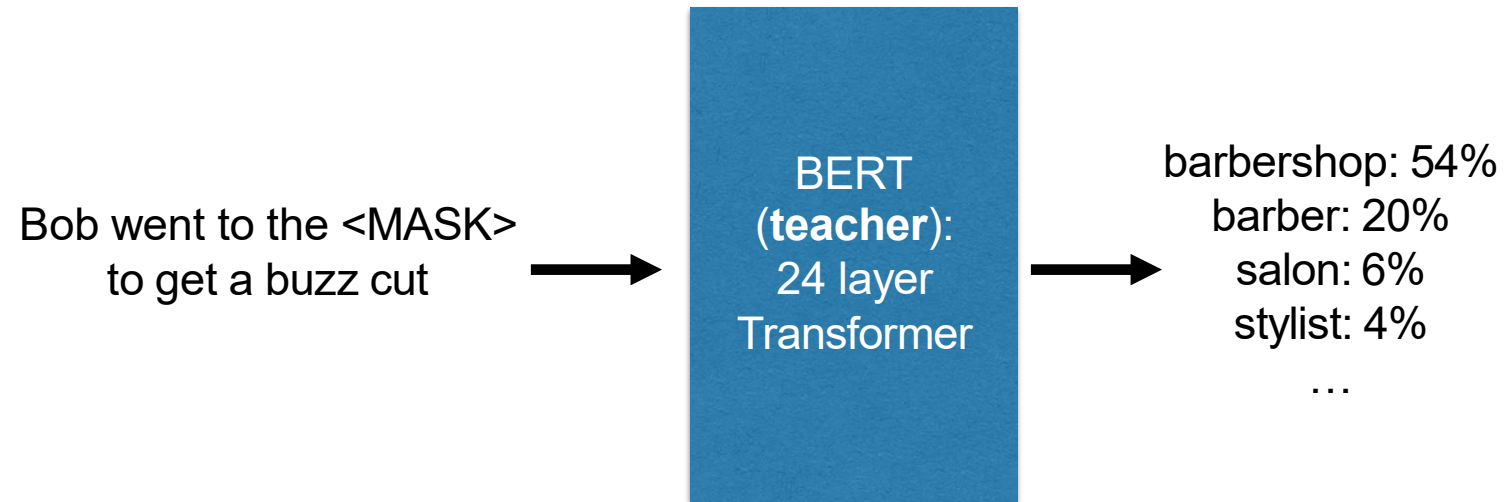
- Domain-specific accurate answering
- Frequent updates of data
- Traceability and explainability of generated content
- Controllable cost
- Privacy protection of data



Drawn by DALL·E-3



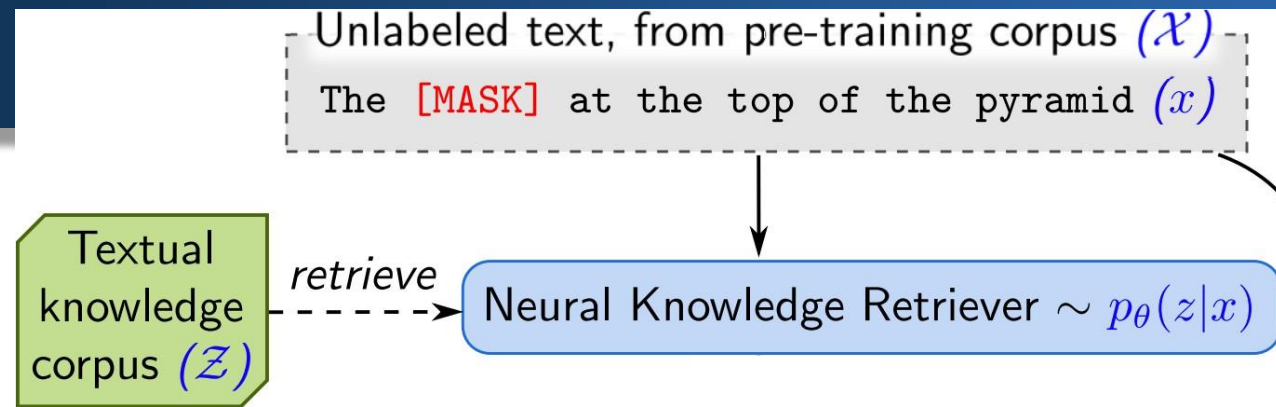
World knowledge is *implicitly* encoded in BERT's parameters! (e.g., that barbershops are places to get buzz cuts)

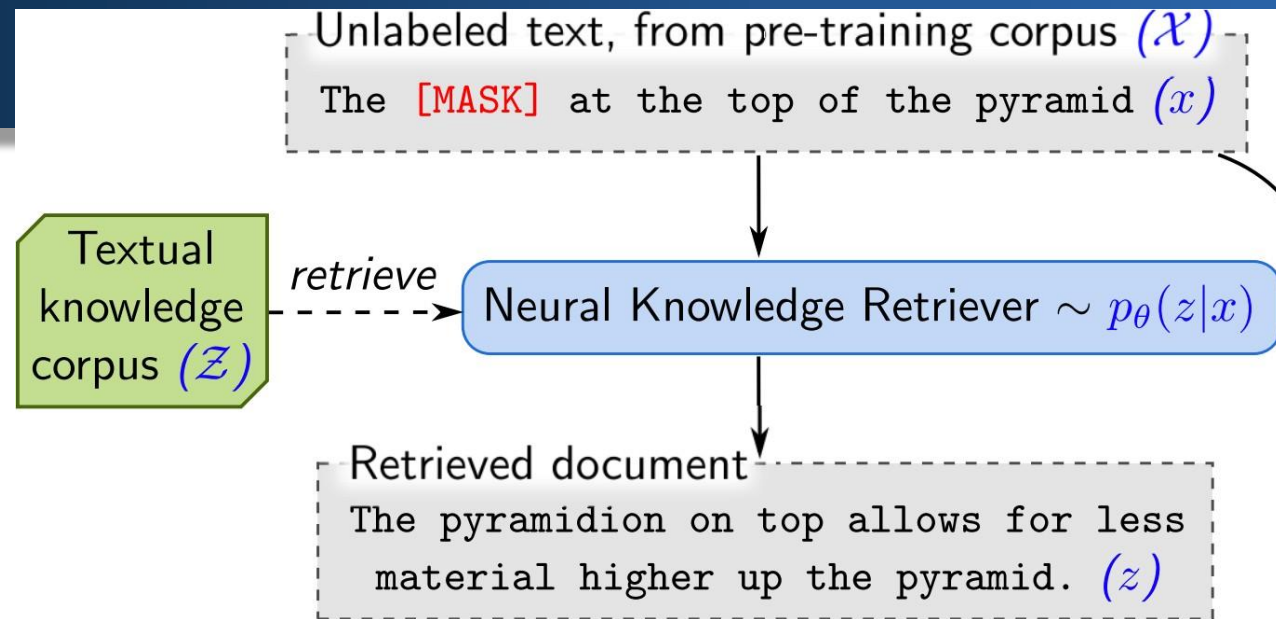


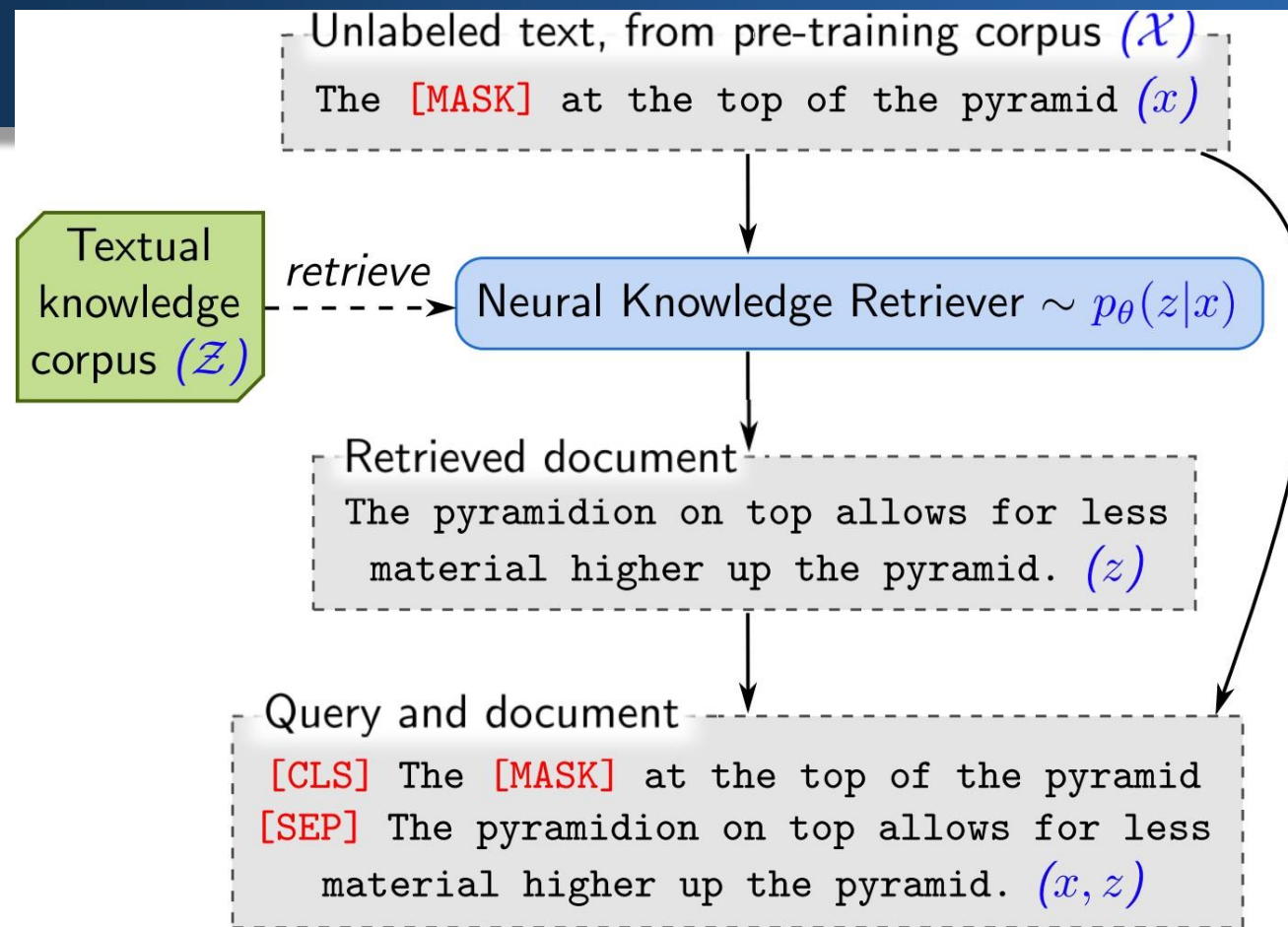
In these language models, the learned world knowledge is stored *implicitly* in the parameters of the underlying neural network. This makes it difficult to determine what knowledge is stored in the network and where. Furthermore, storage space is limited by the size of the network—to capture more world knowledge, one must train ever-larger networks, which can be prohibitively slow or expensive.

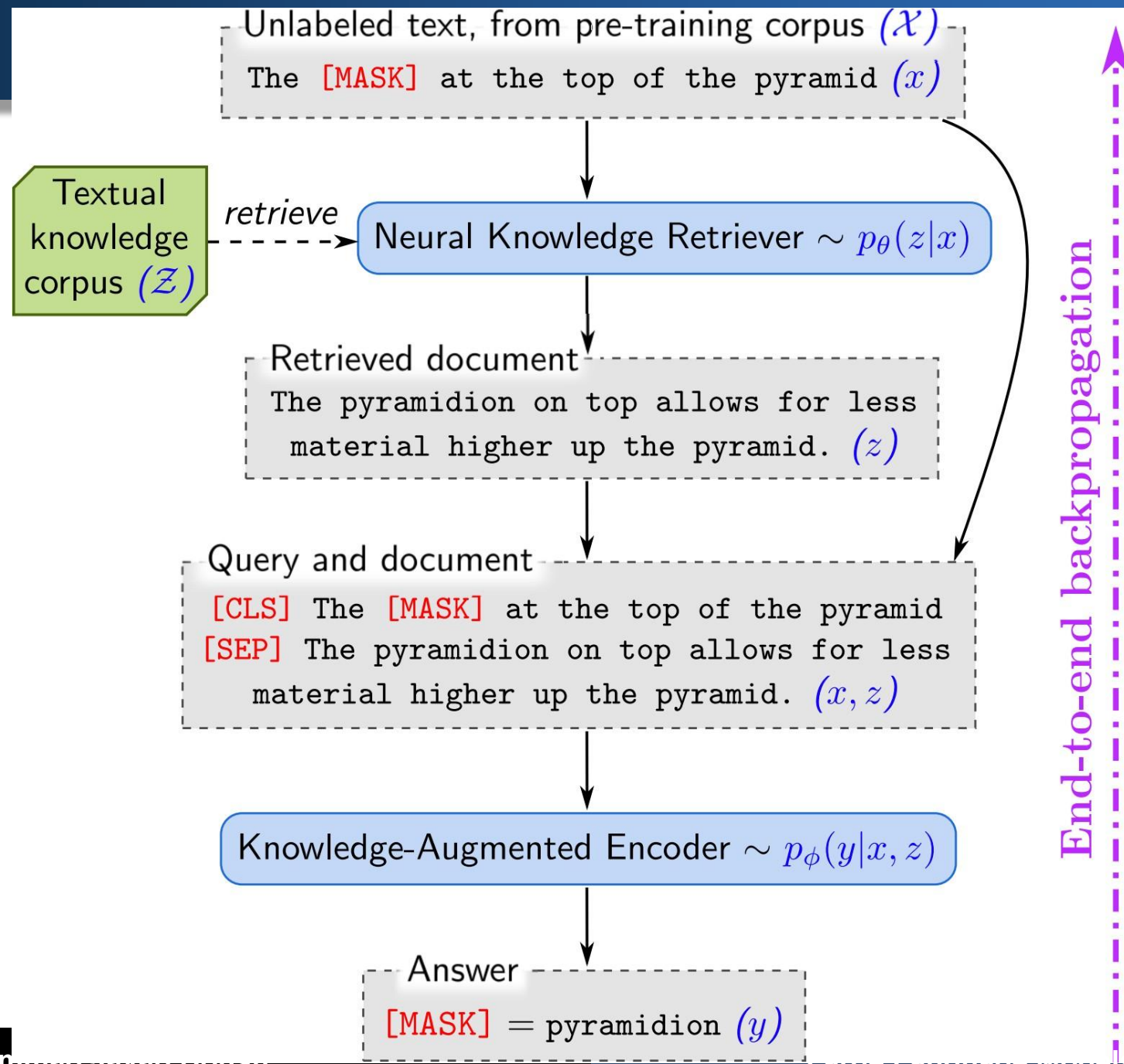
Unlabeled text, from pre-training corpus ($\mathcal{X}$)

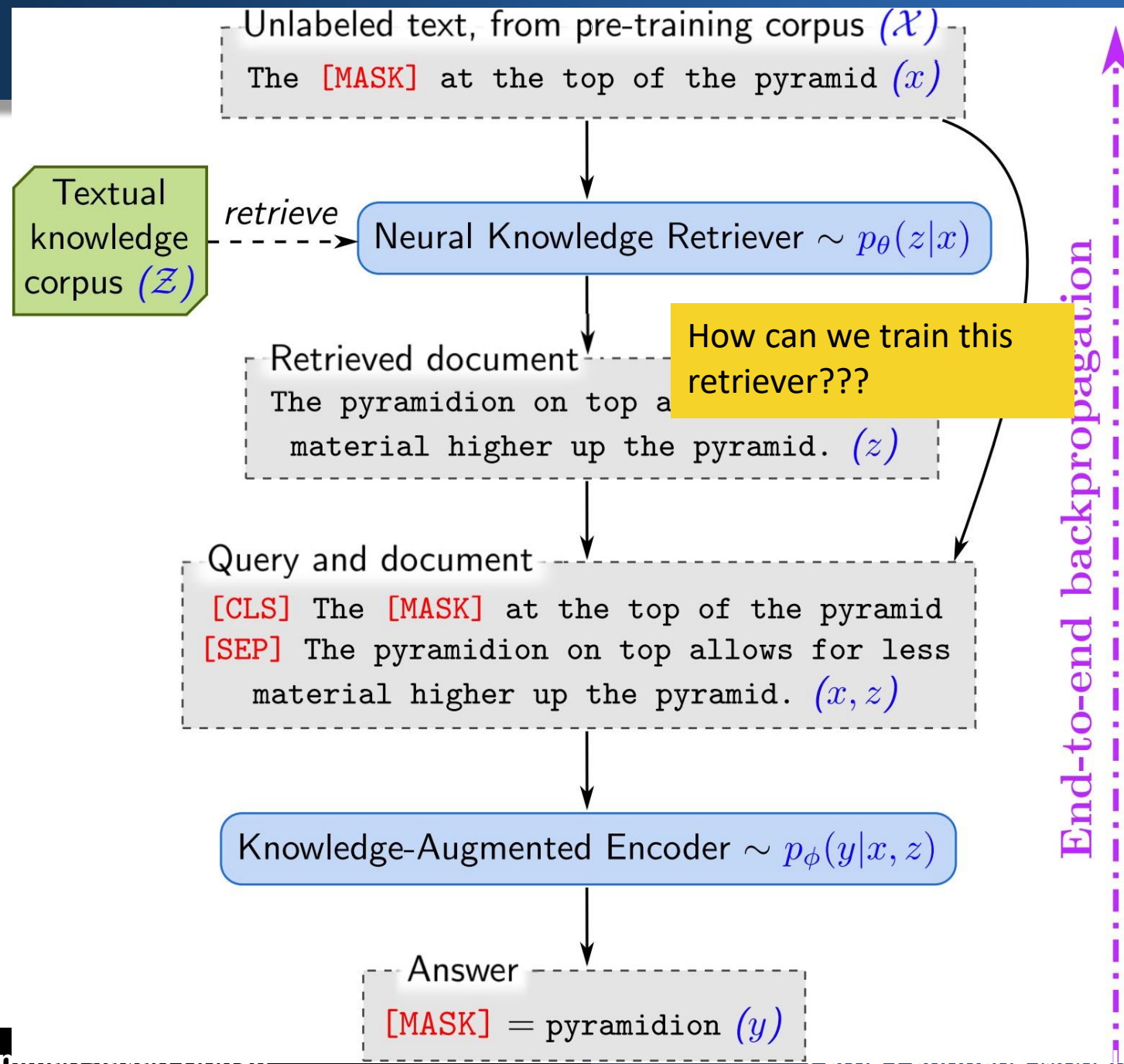
The [MASK] at the top of the pyramid (x)











Sparse Retrieval

□ TF-IDF and BM25 (Robertson, Spärck Jones, et al.)

Given a query Q , containing keywords $q_1, \dots, q_n$, the BM25 score of a document D is:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgl}}\right)}$$

where $f(q_i, D)$ is the number of times that the keyword q_i occurs in the document D , $|D|$ is the length of the document D in words, and avgl is the average document length in the text collection from which documents are drawn. k_1 and b are free parameters, usually chosen, in absence of an advanced optimization, as $k_1 \in [1.2, 2.0]$ and $b = 0.75$.^[3] $\text{IDF}(q_i)$ is the IDF (inverse document frequency) weight of the query term q_i . It is usually computed as:

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$$

where N is the total number of documents in the collection, and $n(q_i)$ is the number of documents containing q_i .

□ Used in DrQA (Chen et al 2017)

Q: How many of Warsaw's inhabitants spoke Polish in 1933?



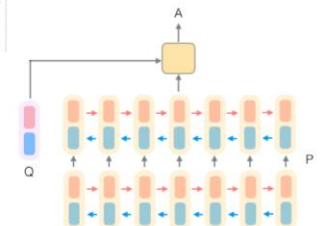
WIKIPEDIA
The Free Encyclopedia

Document
Retriever



Document
Reader

833,500



Dense Retriever

Knowledge Retriever The retriever is defined using a dense inner product model:

$$p(z | x) = \frac{\exp f(x, z)}{\sum_{z'} \exp f(x, z')},$$
$$f(x, z) = \text{Embed}_{\text{input}}(x)^\top \text{Embed}_{\text{doc}}(z),$$

where $\text{Embed}_{\text{input}}$ and $\text{Embed}_{\text{doc}}$ are embedding functions that map x and z respectively to d -dimensional vectors. The *relevance score* $f(x, z)$ between x and z is defined as the inner product of the vector embeddings. The retrieval distribution is the softmax over all relevance scores.

Idea: A vector database

- Based on “semantic similarity” (i.e., cosine score between two embeddings)
- Powered by FAISS (Johnson et al. 2019)

Billion-scale similarity search with GPUs

Jeff Johnson
Facebook AI Research
New York

Matthijs Douze
Facebook AI Research
Paris

Hervé Jégou
Facebook AI Research
Paris

<https://github.com/facebookresearch/faiss>

Feature	FAISS	ChromaDB	Weaviate
🔧 Type	Library (Python/C++)	Embedded Python DB	Full-featured Vector DB
🌐 Interface	Python API	Python API	REST, gRPC, GraphQL, SDKs
🧠 Index Options	Flat, IVF, PQ, HNSW	Basic internal indexing	HNSW, PQ, Hybrid
💾 Persistence	Manual (save/load index)	Automatic with DuckDB	Built-in, scalable
🚀 Speed	Ultra-fast (optimized C++)	Good for local use	Scalable for enterprise
🔒 Security	None (DIY via API layer)	Basic (file access)	Full RBAC, TLS, multitenancy
☁️ Deployment	Local/lib only	Local-first	Cloud-native & Kubernetes
🔍 Hybrid Search	❌	❌	✅ (Vector + keyword + filter)
⚙️ Ideal Use Case	Custom, high-performance apps	Prototypes and small-scale RAG	Production-scale AI search

Source: <https://madhankarthik30.medium.com/faiss-vs-chromadb-vs-weaviate-choosing-the-right-vector-database-for-your-ai-project-62827f423c92>

Evaluation on *open-domain* QA

- ❑ Unlike SQuAD-style QA, in open-domain QA we are only given a question, not a supporting document that is guaranteed to contain the answer
- ❑ Open-domain QA generally has a large *retrieval* component, since the answer to any given question could occur anywhere in a large collection of documents

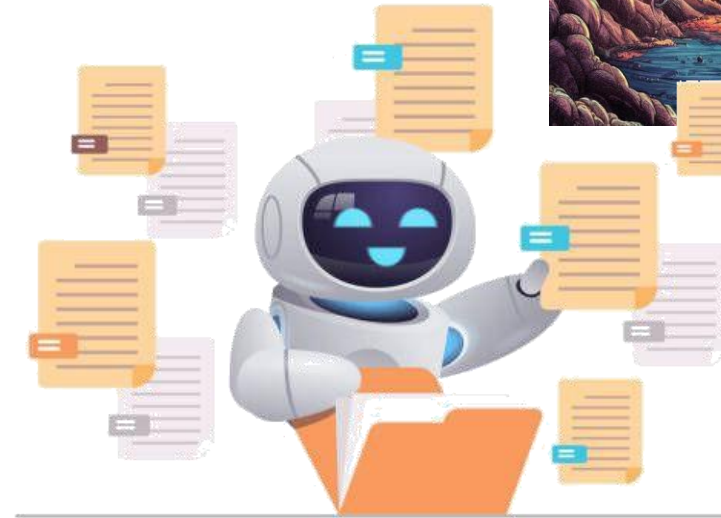
Name	Architectures	Pre-training	NQ (79k/4k)	WQ (3k/2k)	CT (1k /1k)	# params
BERT-Baseline (Lee et al., 2019)	Sparse Retr.+Transformer	BERT	26.5	17.7	21.3	110m
T5 (base) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	27.0	29.1	-	223m
T5 (large) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	29.8	32.2	-	738m
T5 (11b) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	34.5	37.4	-	11318m
DrQA (Chen et al., 2017)	Sparse Retr.+DocReader	N/A	-	20.7	25.7	34m
HardEM (Min et al., 2019a)	Sparse Retr.+Transformer	BERT	28.1	-	-	110m
GraphRetriever (Min et al., 2019b)	GraphRetriever+Transformer	BERT	31.8	31.6	-	110m
PathRetriever (Asai et al., 2019)	PathRetriever+Transformer	MLM	32.6	-	-	110m
ORQA (Lee et al., 2019)	Dense Retr.+Transformer	ICT+BERT	33.3	36.4	30.1	330m
Ours ($\mathcal{X}$ = Wikipedia, $\mathcal{Z}$ = Wikipedia)	Dense Retr.+Transformer	REALM	39.2	40.2	46.8	330m
Ours ($\mathcal{X}$ = CC-News, $\mathcal{Z}$ = Wikipedia)	Dense Retr.+Transformer	REALM	40.4	40.7	42.9	330m

Table 3. An example where REALM utilizes retrieved documents to better predict masked tokens. It assigns much higher probability (0.129) to the correct term, “Fermat”, compared to BERT. (Note that the blank corresponds to 3 BERT wordpieces.)

x : An equilateral triangle is easily constructed using a straightedge and compass, because 3 is a ____ prime.			
(a) BERT	$p(y = \text{“Fermat”} x)$	$= 1.1 \times 10^{-14}$	(No retrieval.)
(b) REALM	$p(y = \text{“Fermat”} x, z)$	$= 1.0$	(Conditional probability with document $z = \text{“257 is ... a Fermat prime. Thus a regular polygon with 257 sides is constructible with compass ...”}$)
(c) REALM	$p(y = \text{“Fermat”} x)$	$= 0.129$	(Marginal probability, marginalizing over top 8 retrieved documents.)

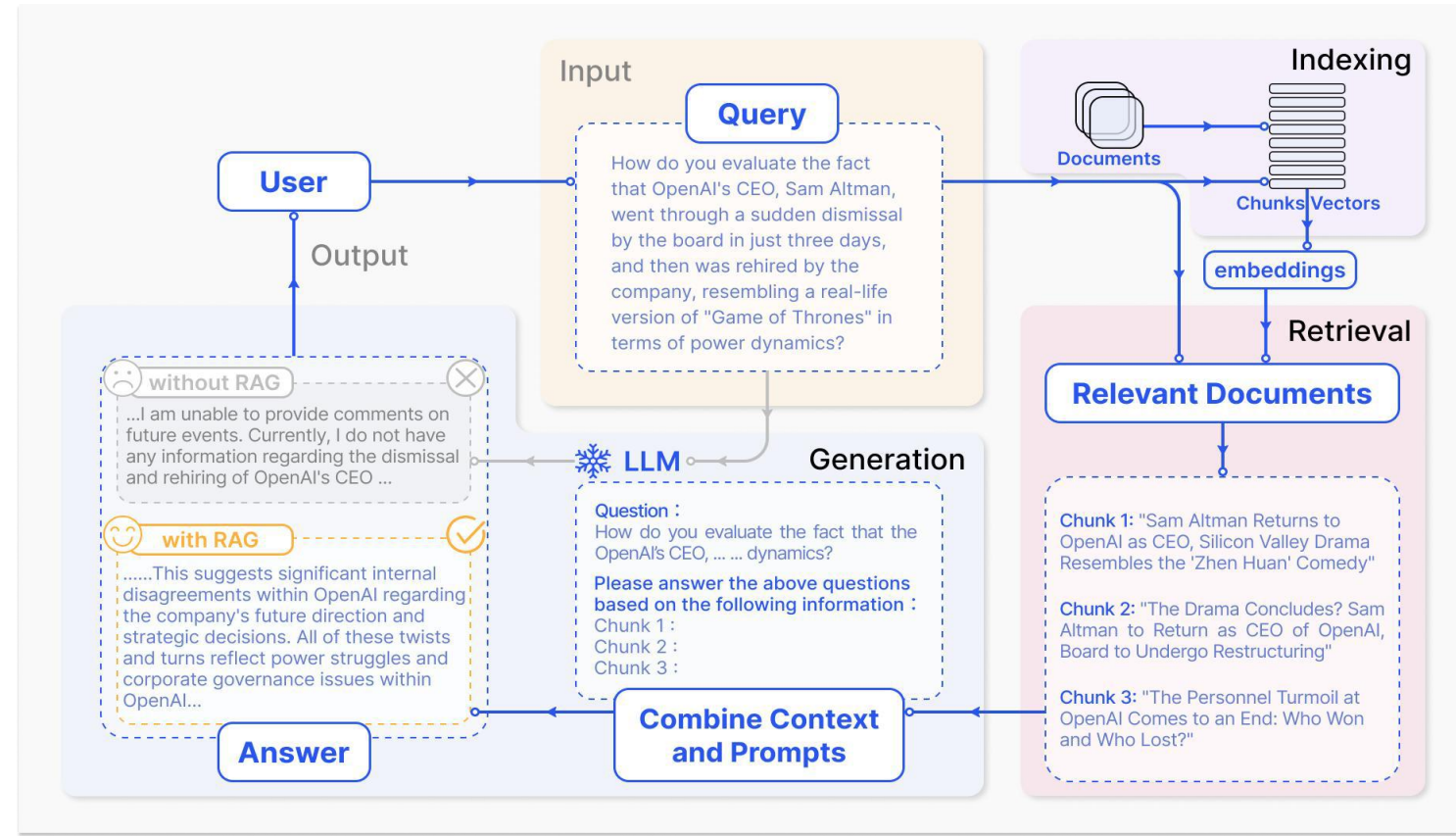
What problems do we have with these

- ❑ Plain IR systems:
 - serves retrieved documents flat -
 - may not interpret queries accurately -
- ❑ Generative LMs (that we know so far):
 - Hallucinations
 - Responses are not specific enough



Retrieval-Augmented Generation (RAG)

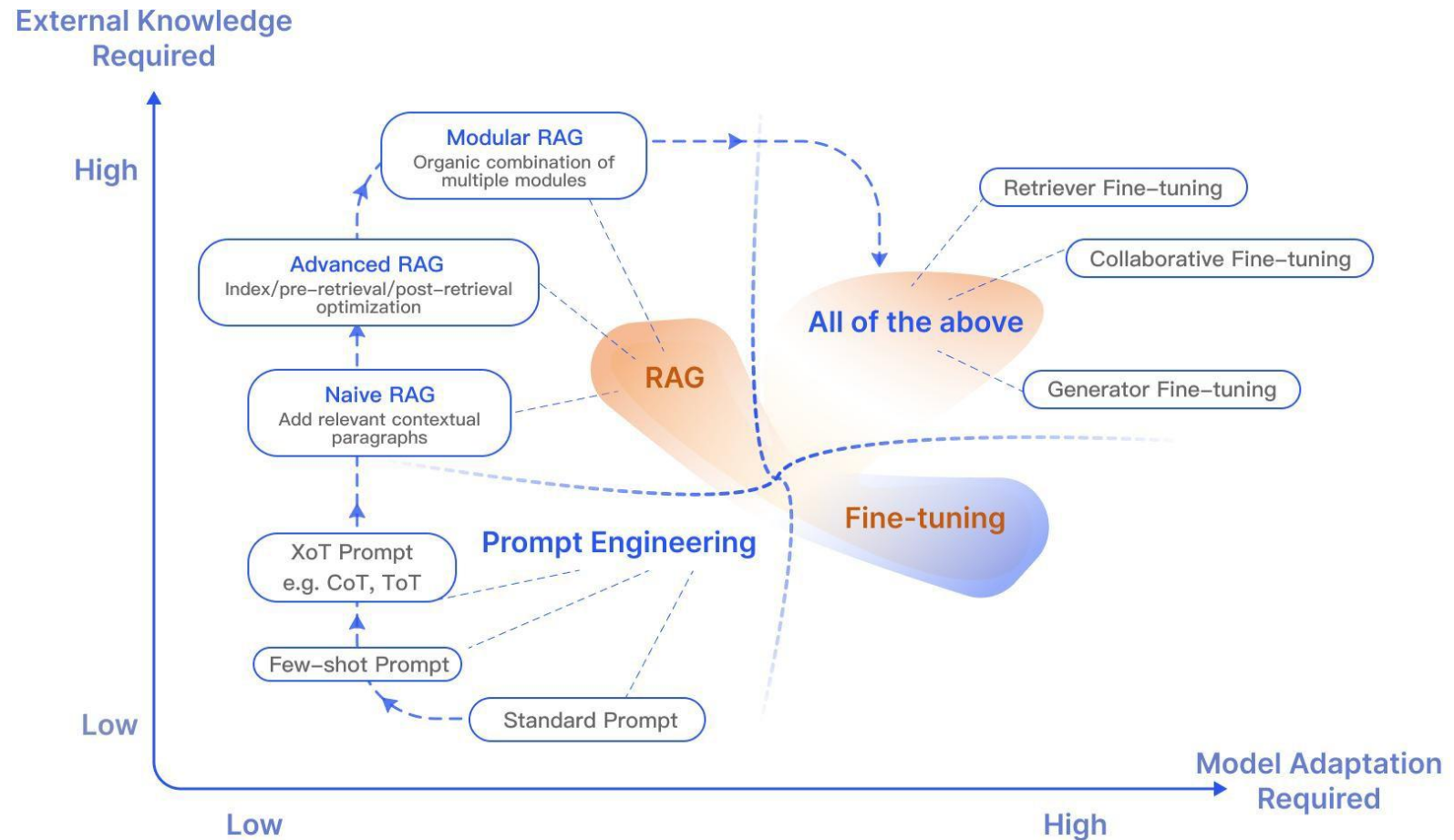
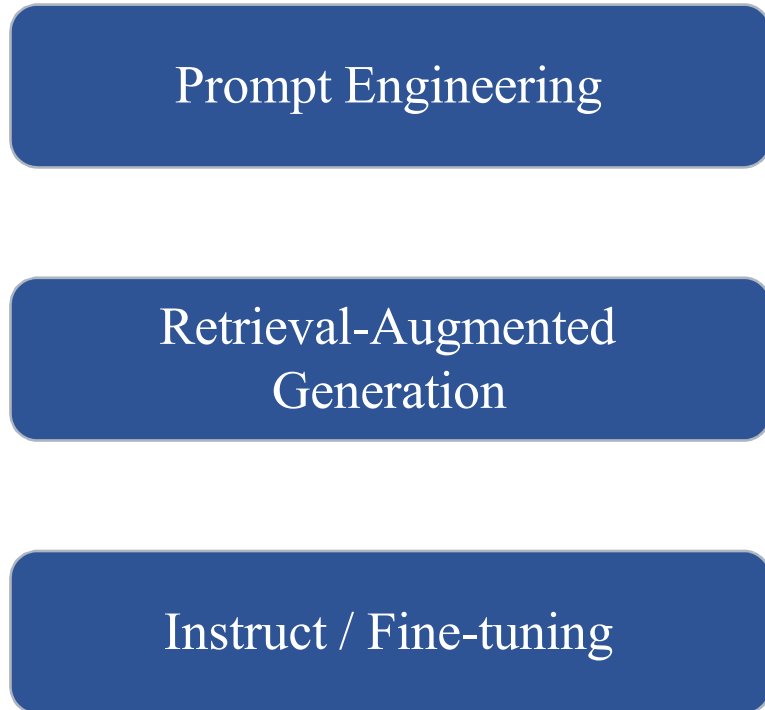
- ❑ When answering questions or generating text, it first **retrieves relevant information** from a large number of documents, and then LLMs generates answers based on this information.
- ❑ By attaching a **external knowledge base**, there is no need to retrain the entire large model for each specific task.
- ❑ The RAG model is especially suitable for **knowledge-intensive** tasks.



A typical case of RAG

Symbolic Knowledge or Parametric Knowledge

- Ways to optimize LLMs.



A typical case of RAG

RAG vs Fine-tuning

Feature Comparison	RAG	Fine-Tuning
Knowledge Updates	Directly updating the retrieval knowledge base ensures that the information remains current without the need for frequent retraining, making it well-suited for dynamic data environments.	Stores static data, requiring retraining for knowledge and data updates.
External Knowledge	Proficient in leveraging external resources, particularly suitable for accessing documents or other structured/unstructured databases.	Can be utilized to align the externally acquired knowledge from pretraining with large language models, but may be less practical for frequently changing data sources.
Data Processing	Involves minimal data processing and handling.	Depends on the creation of high-quality datasets, and limited datasets may not result in significant performance improvements.
Model Customization	Focuses on information retrieval and integrating external knowledge but may not fully customize model behavior or writing style.	Allows adjustments of LLM behavior, writing style, or specific domain knowledge based on specific tones or terms.
Interpretability	Responses can be traced back to specific data sources, providing higher interpretability and traceability.	Similar to a black box, it is not always clear why the model reacts a certain way, resulting in relatively lower interpretability.
Computational Resources	Depends on computational resources to support retrieval strategies and technologies related to databases. Additionally, it requires the maintenance of external data source integration and updates.	The preparation and curation of high-quality training datasets, defining fine-tuning objectives, and providing corresponding computational resources are necessary.
Latency Requirements	Involves data retrieval, which may lead to higher latency.	LLM after fine-tuning can respond without retrieval, resulting in lower latency.
Reducing Hallucinations	Inherently less prone to hallucinations as each answer is grounded in retrieved evidence.	Can help reduce hallucinations by training the model based on specific domain data but may still exhibit hallucinations when faced with unfamiliar input.
Ethical and Privacy Issues	Ethical and privacy concerns arise from the storage and retrieval of text from external databases.	Ethical and privacy concerns may arise due to sensitive content in the training data.

RAG Applications

❑ Scenarios where RAG is applicable:

- Long-tail distribution of data
- Frequent knowledge updates
- Answers requiring verification and traceability
- Specialized domain knowledge
- Data privacy preservation

Q&A

RETRO (Borgeaud et al, 2021)
REALM (Gu et al, 2020)
ATLAS (Izacard et al, 2023)

Fact Checking

RAG (Lewis et al, 2020)
ATLAS (Izacard et al, 2022)
Evi. Generator (Asai et al, 2022a)

Dialog

Blender Bot 3 (Shuster et al, 2022)
Internet-augmented generation (Komeili et al., 2022)

Summary

FLARE (Jiang et al, 2023)

Machine Translation

kNN-MT (Khandelwal et al., 2020)
TRIME-MT (Zhong et al., 2022)

Code Generation

DocPrompting (Zhou et al., 2023)
Natural Prover (Welleck et al., 2022)

Natural Language Inference

kNN-Prompt (Shi et al., 2022)
NPM (Min et al., 2023)

Sentiment analysis

kNN-Prompt (Shi et al., 2022)
NPM (Min et al., 2023)

Commonsense reasoning

Raco (Yu et al, 2022)

RAG Paradigms Shifting

Naïve RAG

❑ Step 1: Indexing

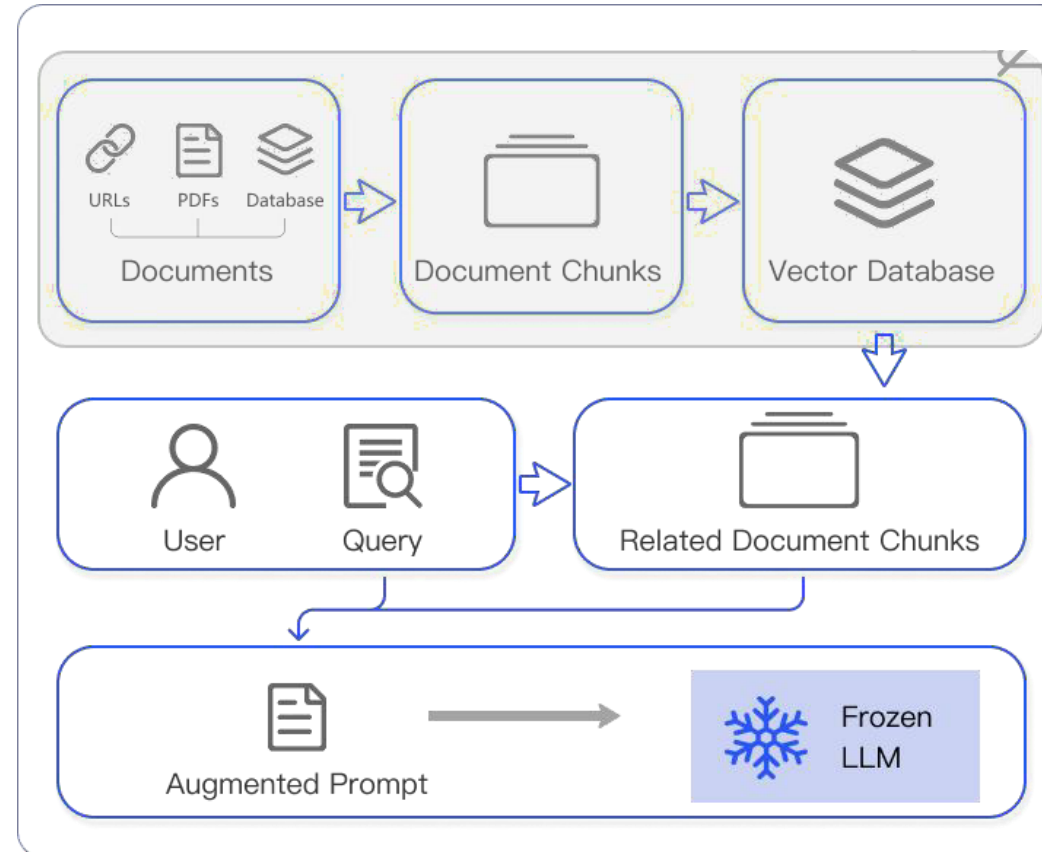
- Divide the document into even chunks, each chunk being a piece of the original text.
- Using the encoding model to generate an embedding for each chunk.
- Store the Embedding of each block in the vector database.

❑ Step 2: Retrieval

- Retrieve the k-most relevant documents using vector similarity search.

❑ Step 3: Generation

- The original query and the retrieved text are combined and input into a LLM to get the final answer



Naive RAG

Advanced RAG

Modular RAG

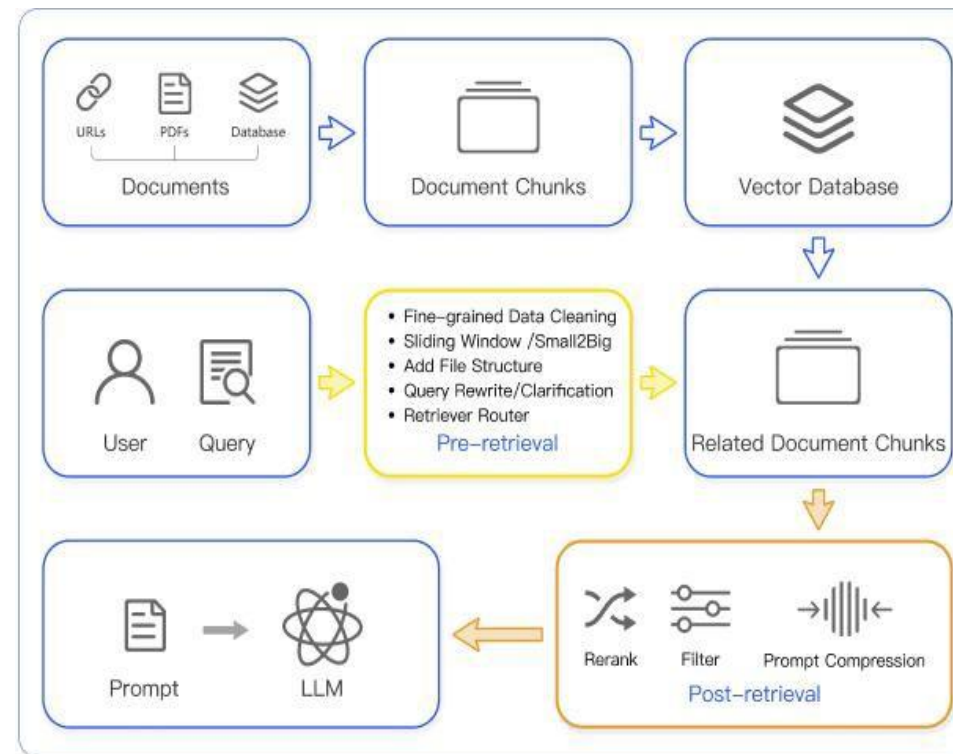
Chunking

- ❑ **Chunk size:** Usually 200-300 tokens (moderate)
- ❑ **Chunking iteration:** Sliding window, with overlap of ~50 tokens
- ❑ **Structure-aware Strategies:** Smarter chunking respects document structure.
- ❑ **Examples:**
 - Splitting on headings and sentences;
 - First breaking at paragraph or section boundaries, and then by sentence;
 - Avoiding mid-sentence splits, etc.

Advanced RAG

Index Optimization → Pre-Retrieval Process → Retrieval →
Post-Retrieval Process → Generation

- ❑ **Optimizing Data Indexing:**
sliding window, fine-grained segmentation, adding metadata
- ❑ **Pre-Retrieval Process:** retrieve routes, summaries, rewriting, and confidence judgment
- ❑ **Post-Retrieval Process:**
reorder, filter content retrieval



Naive RAG

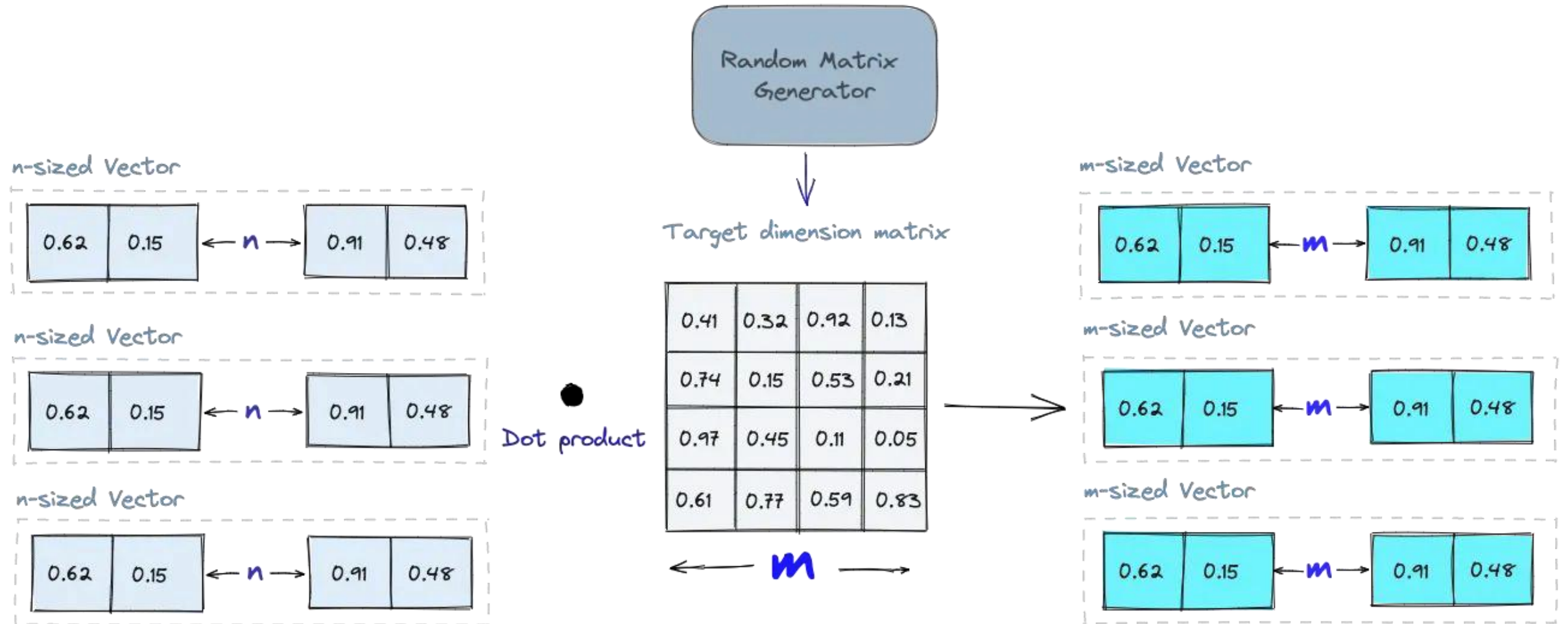
Advanced RAG

Modular RAG

Indexing

- ❑ Constructing the vector database from the documents (embeddings) via certain indexing algorithms.
 - Indexing maps the vectors to a data structure that will enable faster searching.
 - That usually means compressing the vectors to a lower / smaller dimensional space.
 - Seems familiar?

Indexing: Random Projection

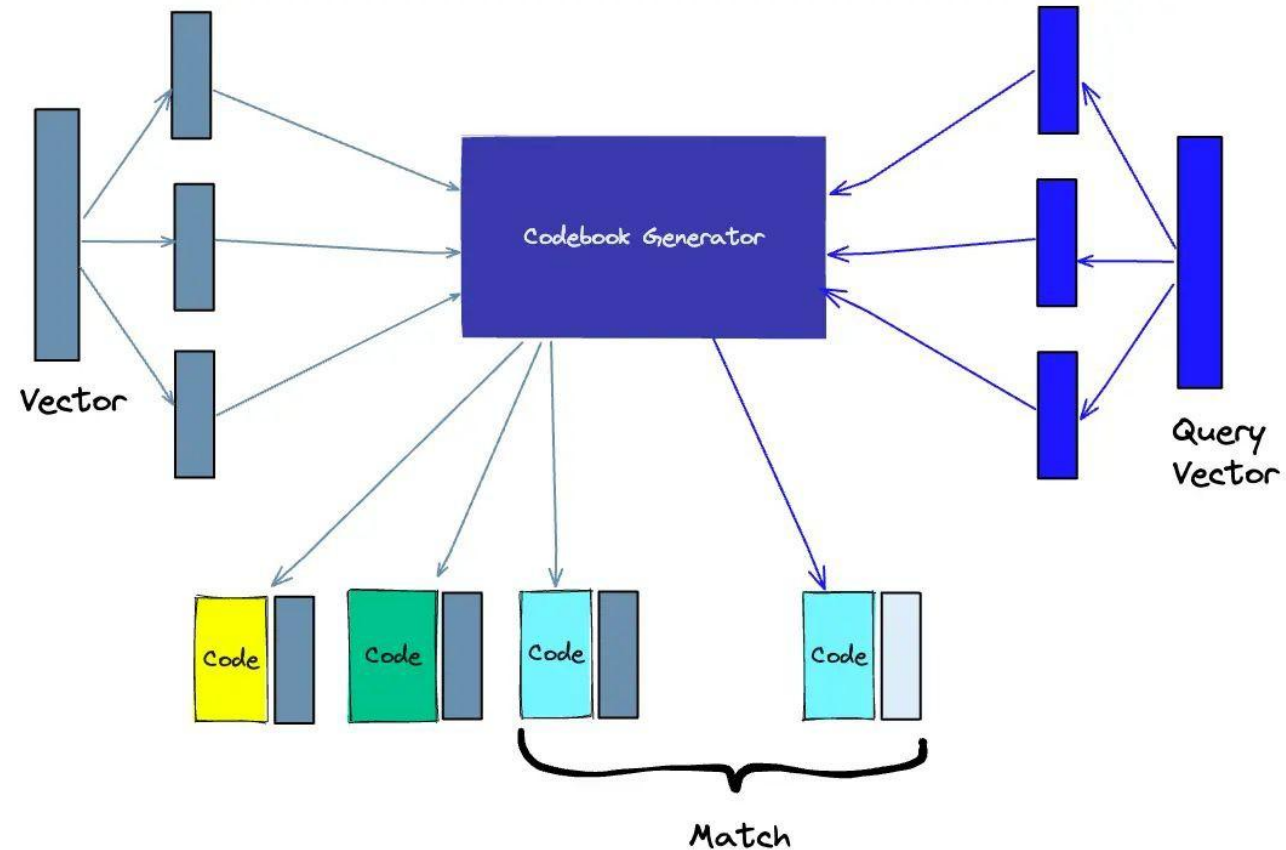


(Figure borrowed from Pinecone)

Indexing: Product Quantization (PQ)

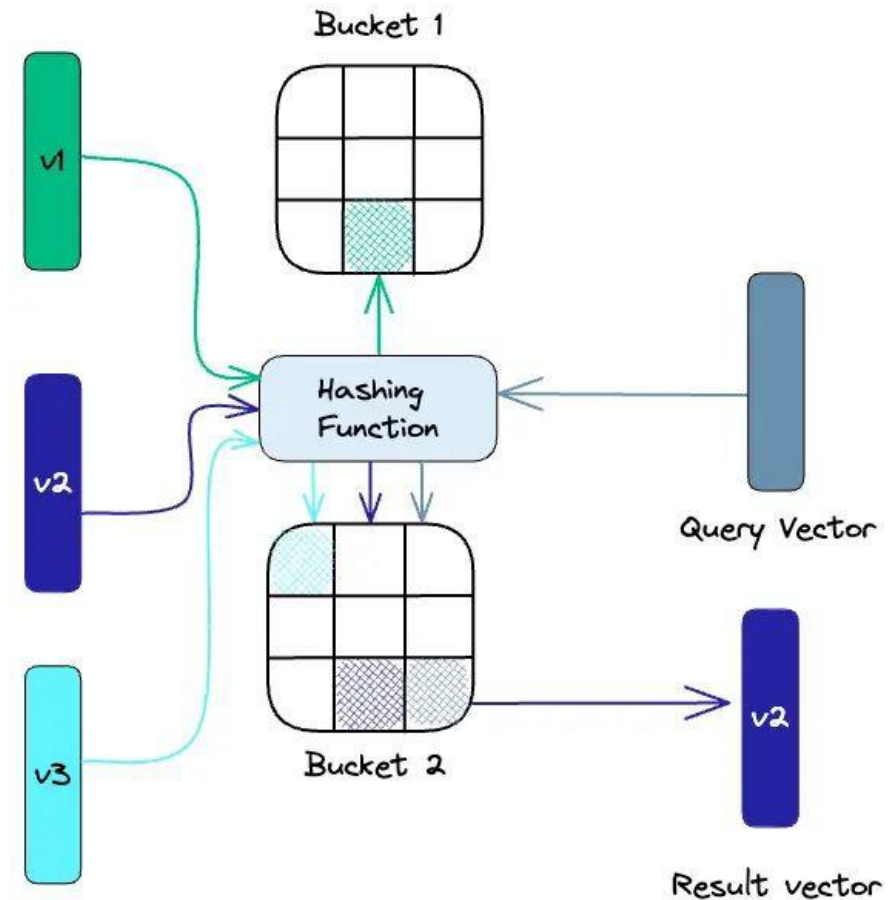
□ General Process:

- **Splitting:** chop vectors into segments;
- **Training:** build a “codebook” for each segment
- **Encoding:** assign a specific code to each segment



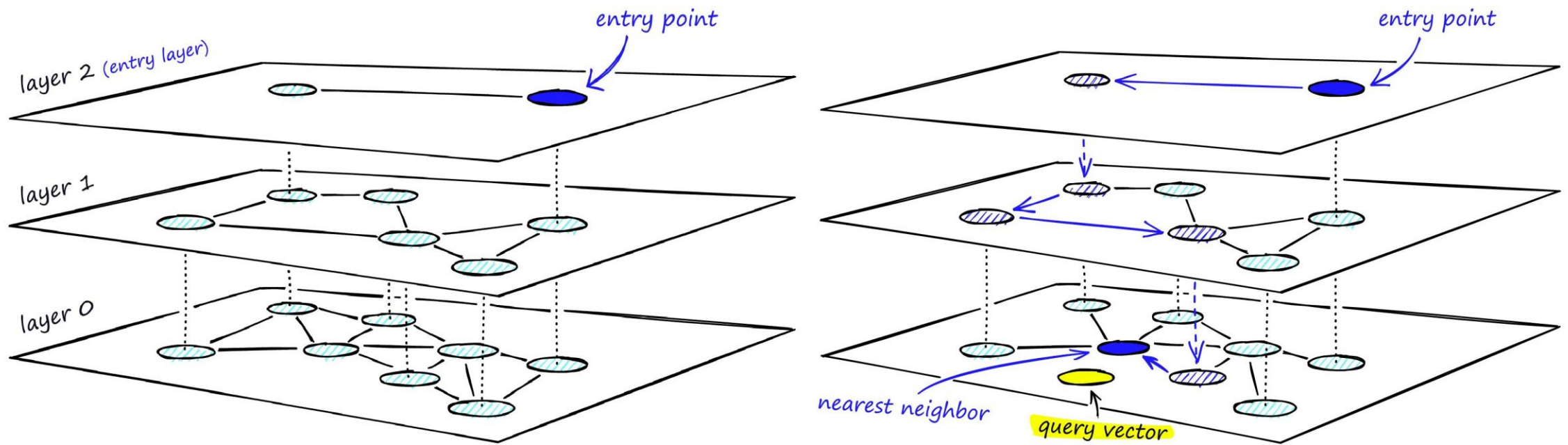
Indexing: Locality-Sensitive Hashing (LSH)

- ❑ LSH maps similar vectors into “buckets” using a set of hashing functions.
- ❑ The same set of hashing functions is used for fetching the query.



(Figure borrowed from Pinecone)

Indexing: Hierarchical Navigable Small Worlds (HNSW)



(Figure borrowed from Pinecone)

Retrieving (1)

- ❑ For each of the aforementioned algorithms, we need a way to fetch context from the vector database that is specific to how indexing was done:
 - **Random Projection:** Compute similarity between projected vectors;
 - **PQ:** Match codebooks;
 - **LSH:** First map query to a “bucket”, then find the closest matches within that “bucket”;
 - **HNSW:** Approximate Nearest Neighbour (ANN)

Retrieving (2)

Algorithmic Strategy	Core Paradigm	Main Optimization Goal	Memory Cost	Search Speed	Best Used For...
Random Projection	Matrix Math / Dimensionality Reduction	Speed up math by shrinking vector size	Low to Medium	Fast	Pre-processing massive, extremely sparse datasets.
PQ	Lossy Compression / Quantization	Shrink database size to fit in RAM	Extremely Low (Best)	Fast (via Lookups)	Massive datasets where RAM costs are the strict limiting constraint.
LSH	Probabilistic Hashing / Bucketing	Limit the search scope to a single bucket	Medium to High	Fast	Discrete distance metrics or sparse document matching pipelines.
HNSW	Proximity Graphs	Maximize accuracy and retrieval speed	High	Extremely Fast (Best)	Mission-critical applications requiring the highest possible accuracy and low latency.

Prompting

- ❑ Other than the “prompt engineering” on the natural language level, we also have to figure out the exact context we are presenting to the LLM.

- ❑ Strategies include:
 - **Hierarchical chunks:** For retrieved chunks, further compute the cosine similarity between those and the query;
 - **Context Window:** Extend retrieved chunks to include context before and after the chunks.
 - **Hybridization with classic IR system:** Remember systems like BM25? We can create a simple mixture model that unifies their results with our RAG results.

RAG Evaluation: Retrieval

- Context Precision: To compute, we need query + contexts + reference answer:

$$\text{Context Precision@}K = \frac{\sum_{k=1}^K (\text{Precision@}k \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}}$$

$$\text{Precision@}k = \frac{\text{true positives@}k}{(\text{true positives@}k + \text{false positives@}k)}$$

Where K is the total number of chunks in `contexts` and $v_k \in \{0, 1\}$ is the relevance indicator at rank k .

- Context Recall: To compute, we need contexts + ground truth (GT) answers:

$$\text{Context Recall} = \frac{|\text{GT sentences that can be attributed to context}|}{|\text{sentences in GT}|}$$

Context Precision (example)

- **Question:** Where is France and what is its capital?
- **Ground truth:** France is in Western Europe and its capital is Paris.
- **High context precision:** ["France, in Western Europe, encompasses medieval cities, alpine villages and Mediterranean beaches. Paris, its capital, is famed for its fashion houses, classical art museums including the Louvre and monuments like the Eiffel Tower", "The country is also renowned for its wines and sophisticated cuisine. Lascaux's ancient cave drawings, Lyon's Roman theater and the vast Palace of Versailles attest to its rich history."]

$$P @ 1 = 1, P@2 = 0.5 \Rightarrow CP @ 2 = (1 \times 1 + 0.5 \times 0) / 1 = 1$$

- **Low context precision:** ["The country is also renowned for its wines and sophisticated cuisine. Lascaux's ancient cave drawings, Lyon's Roman theater and", "France, in Western Europe, encompasses medieval cities, alpine villages and Mediterranean beaches. Paris, its capital, is famed for its fashion houses, classical art museums including the Louvre and monuments like the Eiffel Tower"]

$$P @ 1 = 0, P@2 = 0.5 \Rightarrow CP @ 2 = (0 \times 0 + 0.5 \times 1) / 1 = 0.5$$

Context Recall (example)

- ❑ **Question:** Where is France and what is its capital?
- ❑ **Ground truth:** France is in Western Europe and its capital is Paris.
- ❑ **High context recall:** France, in Western Europe, encompasses medieval cities, alpine villages and Mediterranean beaches. Paris, its capital, is famed for its fashion houses, classical art museums including the Louvre and monuments like the Eiffel Tower.

$$CR = 2 / 2 = 1.0$$

- ❑ **Low context recall:** France, in Western Europe, encompasses medieval cities, alpine villages and Mediterranean beaches. The country is also renowned for its wines and sophisticated cuisine. Lascaux's ancient cave drawings, Lyon's Roman theater and the vast Palace of Versailles attest to its rich history.

$$CR = 1 / 2 = 0.5$$

RAG Evaluation: Generation

- ❑ **Faithfulness:** We are NOT looking for the veracity of the claims, but whether the LLM has responded based on the context:

$$\text{Faithfulness score} = \frac{|\text{Number of claims in the generated answer that can be inferred from given context}|}{|\text{Total number of claims in the generated answer}|}$$

- ❑ **Answer Relevancy:** Compare the embeddings between generated reverse question and the original query.

$$\text{Answer Relevancy} = \frac{1}{N} \sum_1^N \cos(E_{g_i}, E_o)$$

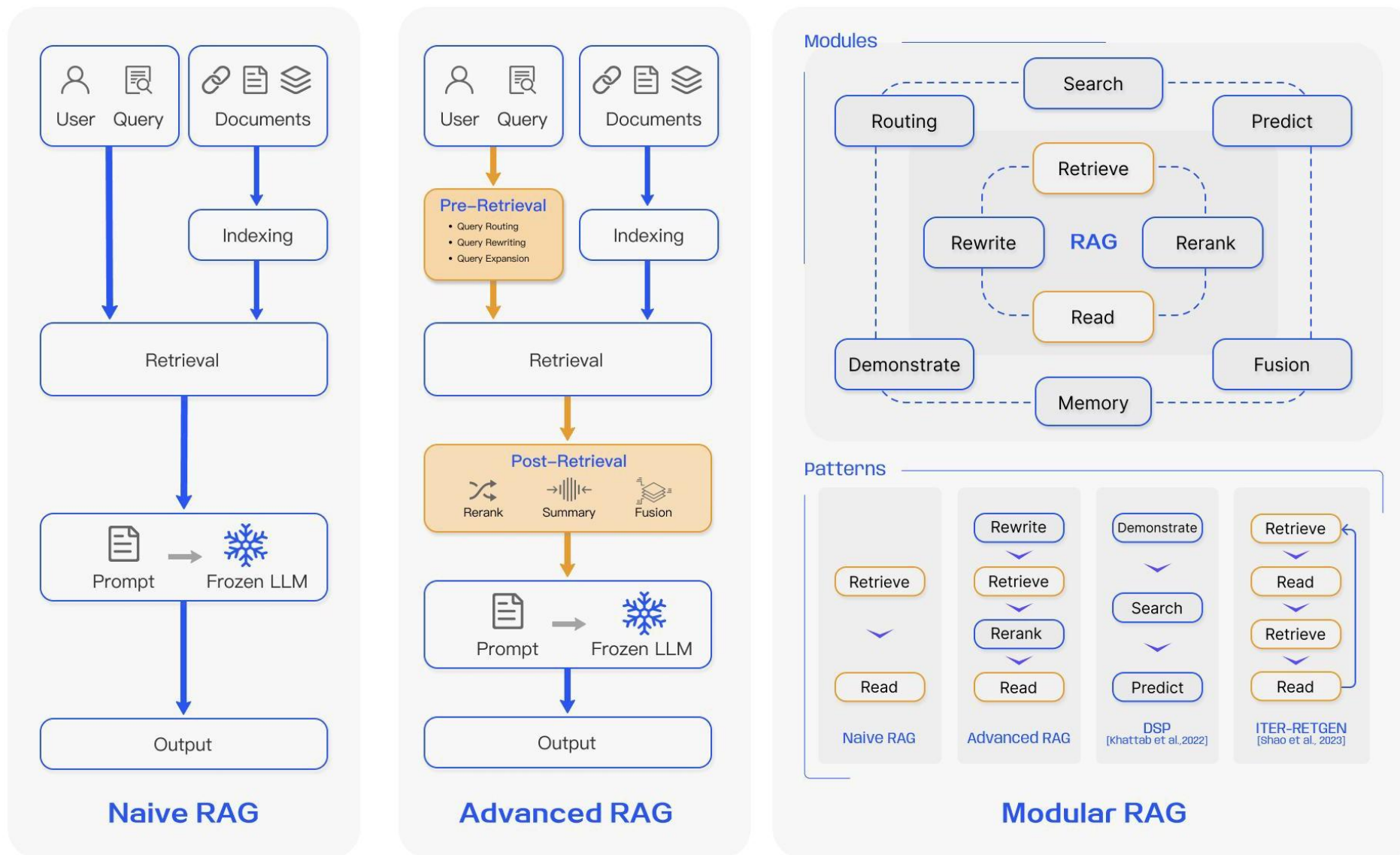
Faithfulness (example)

- ❑ **Question:** Where and when was Einstein born?
- ❑ **Context:** Albert Einstein (born 20 March 1879) was a German-born theoretical physicist, widely held to be one of the greatest and most influential scientists of all time
- ❑ **High faithfulness answer:** Einstein was born in Germany on 20th March 1879.
- ❑ **Low faithfulness answer:** Einstein was born in Germany on 14th March 1879.
- ❑ **Please note:** Albert Einstein was actually born on 14th March 1879, factually.

Answer Relevancy (example)

- ❑ **Question:** Where is France and what is its capital?
- ❑ **Low relevance answer:** France is in western Europe.
- ❑ **High relevance answer:** France is in western Europe and Paris is its capital.

Comparison of RAG Paradigms



The three key questions of RAG

What to retrieve ?

- Token
- Phrase
- Chunk
- Paragraph
- Entity
- Knowledge graph

When to retrieve?

- Single search
- Each token
- Every N tokens
- Adaptive search

How to use the retrieved information?

- Input/Data Layer (prompt level)
- Model/Intermediate Layer (Architecture-Level Fusion)
- Output/Prediction Layer (Generation-Level Fusion)

Other Issues

Augmentation stage:

- Pre-training
- Fine-tuning
- Inference

Retrieval choice:

- BERT
- RoBERTa
- BGE
-

Model
Collaboration



Scale
selection

Generation choice:

- GPT
- Llama
- T5
-

Overview of RAG Development

